

TALLER PRÁCTICO DE TYPESCRIPT

Fundamentos de Arrays, Map y Filter

Libro de Ejercicios para Estudiantes

Universidad Alexander von Humboldt

Ingeniería de Software

Algoritmos - 2026

Introducción

Bienvenido a este taller de TypeScript. Este NO es un libro de soluciones, sino una guía para que TÚ construyas tu propio conocimiento. Cada ejercicio está diseñado para desafiarte a pensar como ingeniero de software y resolver problemas reales.

Cómo Usar Este Taller

1. Lee atentamente la descripción del problema
2. Estudia los requisitos y las preguntas guía
3. Planifica tu solución en papel o pseudocódigo
4. Escribe el código TypeScript en tu editor
5. Prueba tu solución y ajusta si es necesario

Conceptos Fundamentales

1. Arrays de Objetos

Un array de objetos es una colección donde cada elemento es un objeto con propiedades específicas. En TypeScript, defines la estructura con una interfaz. Piensa en ello como una tabla: cada fila es un objeto, y cada columna es una propiedad.

2. El método `map()`

`map()` transforma cada elemento de un array. Recibe una función que define cómo transformar cada elemento, y retorna un NUEVO array. Usos: extraer propiedades, cambiar formatos, aplicar cálculos, crear resúmenes.

3. El método `filter()`

`filter()` crea un nuevo array con SOLO los elementos que cumplen una condición. Usos: buscar elementos, eliminar no válidos, separar en categorías, contar elementos.

4. El método `reduce()`

`reduce()` convierte un array en un solo valor acumulando resultados. Usos: sumar precios, contar elementos, crear objetos resumen, agrupar datos.



Ejercicio 1: Sistema de Cajero

Automático

Nivel: Principiante | Tiempo estimado: 45 minutos

Descripción del Problema

Debes crear un sistema de cajero automático que simule las operaciones básicas bancarias. El sistema debe mantener un registro completo de todas las transacciones realizadas y permitir generar reportes detallados. Este es tu primer desafío: debes diseñar una solución que maneje estado mutable (el saldo cambia) mientras mantienes un historial inmutable de lo que sucedió.

Requisitos Funcionales

Operación 1: CONSULTAR SALDO

- Mostrar el saldo actual de la cuenta
- NO debe modificar el saldo ni crear una transacción

Operación 2: DEPOSITAR DINERO

- Aumentar el saldo con el monto depositado
- Registrar la transacción en el historial
- Validar que el monto sea positivo

Operación 3: RETIRAR DINERO

- Disminuir el saldo con el monto retirado
- Registrar la transacción
- RECHAZAR si hay fondos insuficientes

Operación 4: ESTADO DE CUENTA

- Mostrar TODAS las transacciones realizadas
- Mostrar el saldo resultante después de cada transacción
- Calcular estadísticas: total depositado, total retirado
- Usar `map()` para formatear cada transacción

Estructura de Datos Sugerida

Necesitarás al menos DOS estructuras:

1. Una interfaz para CADA TRANSACCIÓN que contenga:

- Un identificador único (número secuencial)
- Tipo de operación (depósito, retiro, consulta)
- El monto involucrado
- La fecha y hora
- El saldo resultante después de la operación

2. Una interfaz para la CUENTA BANCARIA que contenga:

- Nombre del titular
- Saldo actual
- Un array de todas las transacciones

Preguntas Guía para tu Solución

- ¿Cómo asignarás IDs únicos a las transacciones?
- ¿Dónde verificarás si hay fondos suficientes para un retiro?
- ¿Cómo usarías `map()` para transformar el array de transacciones en un resumen legible?
- ¿Cómo usarías `filter()` para separar depósitos de retiros y calcular totales?
- ¿Qué pasaría si intentas retirar más dinero del que tienes? ¿Cómo lo manejarías?



Ejercicio 2: Sistema de Reserva de Hotel

Nivel: Intermedio | Tiempo estimado: 90 minutos

Descripción del Problema

Diseña un sistema completo de reservas para un hotel con tres categorías de habitaciones. El desafío aquí es más complejo: debes manejar precios dinámicos que cambian según la cantidad de noches, aplicar descuentos automáticos, y generar reportes de ocupación. Este ejercicio te enseñará a trabajar con lógica condicional para calcular precios.

Las Tres Categorías de Habitaciones

- ECONÓMICA: \$80 por noche | Servicios básicos
- ESTÁNDAR: \$150 por noche | TV, Wi-Fi, minibar
- SUITE: \$300 por noche | Lujo completo

Sistema de Descuentos Progresivos

Cuanto más noches reserve un cliente, mayor descuento recibe:

- 1-2 noches: 0% descuento
- 3-5 noches: 5% descuento
- 6-10 noches: 10% descuento
- 11+ noches: 15% descuento

Tu sistema debe calcular automáticamente cuál es el descuento según la cantidad de noches.

Requisitos Funcionales

1. Crear una reserva con validación del tipo de habitación
2. Calcular precio: $(\text{tarifa} \times \text{noches}) \times (1 - \text{descuento})$
3. Guardar la reserva con todos sus datos
4. Generar reportes que muestren: ingresos por tipo, ocupación, descuentos totales
5. Usar `map()` para transformar reservas en datos de reporte

Preguntas Guía para tu Solución

- ¿Cómo estructurarías la lógica para determinar el descuento según noches?
- ¿Dónde guardarías la información de las habitaciones disponibles?
- ¿Cómo usarías `filter()` para agrupar reservas por tipo de habitación?
- ¿Cómo combinarías `map()` con `reduce()` para calcular ingresos totales?
- ¿Qué información necesitarías mostrar en un reporte de ocupación?



Ejercicio 3: Sistema de Alquiler de Lavadoras

Nivel: Intermedio | Tiempo estimado: 75 minutos

Descripción del Problema

Crea un servicio de alquiler de lavadoras automáticas por horas. Este sistema es similar al del hotel, pero con diferencias importantes: el inventario (lavadoras) cambia de estado (disponible/ocupada), y el cálculo de precio es simple pero con descuentos progresivos. Aquí aprenderás a manejar estado compartido: mientras una lavadora está alquilada, no puede ser alquilada por otro cliente.

Precio Base: \$2 por hora

- ## Requisitos Funcionales

17. 1. Cuando un cliente llega, asignarle un número de turno único (secuencial)
18. 2. Guardar el cliente en la fila con estado 'esperando'

19. 3. Atender al PRIMER cliente de la fila (FIFO). Cambiar su estado a 'atendido'
20. 4. Registrar cuánto tiempo esperó cada cliente antes de ser atendido
21. 5. Registrar cuánto tiempo tardó la atención de cada cliente
22. 6. Mostrar estadísticas: cuánta gente espera, promedio de espera, tiempo promedio de atención
23. 7. Usar `filter()` para obtener clientes por estado, `map()` para transformar datos, `reduce()` para calcular promedios

Preguntas Guía para tu Solución

- ¿Cómo garantizas que cada turno tiene un número único?
- ¿Cómo implementas FIFO? (El primero que entra debe ser el primero atendido)
- ¿Cómo calculas cuánto tiempo esperó un cliente?
- ¿Cómo obtendrías el tiempo PROMEDIO de espera usando `reduce()`?
- ¿Cómo mostrarías únicamente los clientes que están esperando actualmente?
- ¿Cómo contarías cuántas personas han completado su trámite?



Ejercicio 5: Sistema de Venta de Boletas de Cine

Nivel: Avanzado | Tiempo estimado: 120 minutos

Descripción del Problema

Este es tu proyecto de cierre: un sistema completo de venta de entradas de cine. Es complejo porque combina todo lo aprendido: múltiples estructuras de datos (películas, horarios, salas), validaciones complejas, gestión de disponibilidad (asientos), precios diferenciados, y reportes avanzados. Este tipo de sistema existe en aplicaciones reales como Fandango y Cinemark.

Elementos del Sistema

PELÍCULAS DISPONIBLES (al menos 3):

- Cada película tiene: ID, título, género, duración, horarios disponibles
- Cada sala tiene capacidad de 100 asientos

TIPOS DE BOLETAS Y PRECIOS:

- Adulto: \$10
- Estudiante: \$7
- Niño: \$5

Requisitos Funcionales

24. 1. El cliente selecciona película, horario, tipo de boleta y cantidad
25. 2. Validar que la película exista
26. 3. Validar que el horario esté disponible para esa película
27. 4. Contar cuántas boletas se han vendido para ese horario
28. 5. Validar que haya suficientes asientos disponibles
29. 6. Calcular: $\text{total} = (\text{precio unitario} \times \text{cantidad})$
30. 7. Registrar la venta
31. 8. Generar reportes: ingresos totales, ocupación por película, películas populares ($>70\%$)

Preguntas Guía para tu Solución

- ¿Cómo buscarías la película solicitada? ¿`filter()` o `find()`?
- ¿Cómo verificarías que el horario solicitado existe para esa película?
- ¿Cómo contarías cuántas boletas ya se vendieron para un horario específico?
- ¿Dónde guardarías los precios de cada tipo de boleta?
- ¿Cómo calcularías el porcentaje de ocupación de una película?
- ¿Cómo mostrarías solo las películas con ocupación $\geq 70\%$? ¿Qué método usarías?

Conclusión y Próximos Pasos

Felicidades por completar este taller. Has aprendido conceptos fundamentales de TypeScript que son esenciales para cualquier ingeniero de software.

Habilidades Adquiridas

- Diseño e implementación de interfaces TypeScript
- Manipulación avanzada de arrays con map y filter
- Desarrollo de sistemas simulados con lógica compleja
- Generación de reportes y estadísticas
- Buenas prácticas de código limpio y mantenible

Desafíos para Profundizar

32. 1. Cajero: Agrega soporte para múltiples cuentas y transferencias entre cuentas
33. 2. Hotel: Implementa un sistema de cancelaciones y reembolsos
34. 3. Lavadoras: Agrega un sistema de mantenimiento que marca lavadoras como no disponibles
35. 4. Banco: Añade turnos VIP que se atienden antes que los normales
36. 5. Cine: Integra un sistema de cupones y promociones de descuento

Consejos de Programación Profesional

- NOMBRES CLAROS: Las variables deben decir qué son
- FUNCIONES PEQUEÑAS: Cada función debe hacer UNA cosa bien
- COMENTARIOS: Explica el POR QUÉ, no el QUÉ
- TIPOS: Usa interfaces para documentar tu código
- VALIDACIÓN: Siempre verifica que los datos sean válidos
- TESTING: Prueba tu código con muchos casos

¡Sigue aprendiendo, sigue construyendo!

Universidad Humboldt - Facultad de Ingeniería de Software